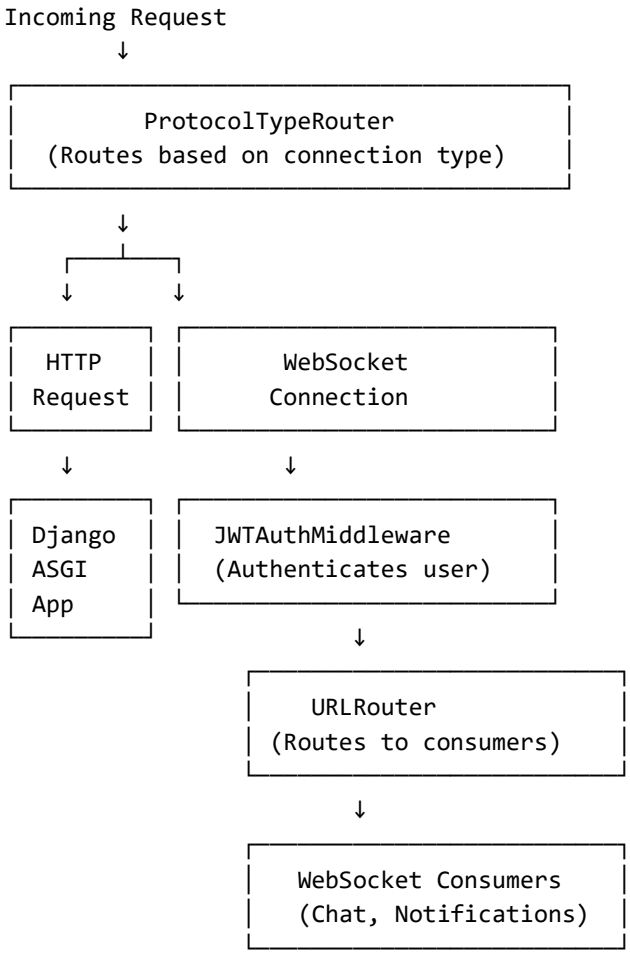
BACKEND SHARE TRAJET(CONFIG) :

Asgi.py: This file is used to handle both regular HTTP requests and websocket connections for real time chat in share trajet

HTTP layer handles normal API requests and websocket layer handles real time chat connections



Without This File	With This File
No real-time chat	✅ Group chats for trips
No real-time notifications	✅ Instant notifications
WebSocket connections fail	✅ WebSockets work

This file **turns our Django app into a real-time application** by adding WebSocket support with authentication, so users can chat and receive instant notifications.

Build_settings.py: this file configures django specifically for the build process when deploying to render and this file is a solution when deploying to render or django and this file creates a temporary lightweight database just for the build process

Feature	Build Settings	Production Settings
Database	SQLite (temp)	PostgreSQL
Cache	In-memory	Redis or database
Email	Console (prints)	SMTP server
Static Files	Collected locally	Served by CDN

```
1. Render starts build
        ↓
2. Loads build_settings.py
        ↓
3. Creates temporary SQLite database
        ↓
4. Runs migrations
   python manage.py migrate
        ↓
5. Collects static files
   python manage.py collectstatic
        ↓
6. Build completes ✅
        ↓
7. Production starts (uses real settings)
```

This file **replaces the real database and cache with lightweight temporary versions** during the build phase so Render can compile our Django app without needing the actual production database.

Settings.py: is the main configuration file for our dz carpooling django project it controls everything about how our application runs

Area	What It Controls
Security	Secret key, allowed hosts, CORS, CSRF
Database	PostgreSQL (local & production)
Real-time	Redis, WebSockets, Channels
API	Authentication, pagination, rate limiting, throttling
Auth	JWT tokens (1h/7d), Google login
Email	SMTP in production, console in dev
Static/Media	Static files, uploaded media

This settings file **configures everything that our Django app needs**—from database connections and authentication to real-time WebSockets, CORS, email, and logging—with separate behaviors for development and production.

Urls.py: this file defines all the URLs or web addresses for our django application mapping each path to the appropriate view or app

The main structure of our URL is :

```
https://yourdomain.com/
    |
    ├── admin/              → Admin panel
```

```
    ├── api/                → API endpoints
    │   ├── schema/         → API documentation (OpenAPI)
    │   ├── docs/           → Swagger UI (interactive docs)
    │   └── redoc/          → ReDoc (alternative docs)
    ├── health/             → Health check (for monitoring)
    └── accounts/           → Google login (Allauth)
```

And the full URL tree will be like this :

```
/
├── admin/                          → Django Admin
├── api/
│   ├── schema/                     → OpenAPI schema (machine-readable)
│   ├── docs/                       → Swagger UI (human-friendly)
│   ├── redoc/                      → ReDoc (alternative docs)
│   └── v1/
│       ├── users/                  → User registration, login, profile
│       ├── trajets/                → List, create, search trips
│       ├── reservations/           → Book, confirm, cancel trips
│       ├── messaging/              → Chat, conversations
│       └── notifications/          → Read, mark as read
├── health/                         → Server status check
└── accounts/                       → Google login
```

This URL configuration **maps every web address to its handler**—admin, API endpoints, documentation, health checks, and Google login—creating the complete routing system for your application.

Wsgi.py: it is the WSGI entry point for our django application it tells web servers how and the WSGI is the WEB SERVER GATEWAY INTERFACE is the standard way python web application communicate with web servers

```
Web Server (Nginx/Apache)
        ↓
    WSGI Server (Gunicorn/uWSGI)
        ↓
    This WSGI file (wsgi.py)
```

```
    ↓
Your Django Application
```

Scenario	Usage
Production	Gunicorn uses it to serve your app
Local testing	`python manage.py runserver` doesn't use it directly
Deployment	Render, Heroku, etc. use it to start your app

So in the end this file creates the WSGI application pbject that web servers use to run our dz carpooling plateform

SAIDI_MAHMOUD_G07